

Connecting a React Frontend (Netlify) to a Node/Express Backend (Render)

Learning Objectives

By the end of this guide, you will be able to:

- Deploy a React/Vite frontend to **Netlify**.
- Deploy a Node/Express backend to **Render**.
- Configure environment variables securely for both ecosystem services.
- Successfully connect your frontend to your live backend.
- Understand why environment variables dynamically change between development and production.

✓ Before You Begin

Ensure you have setup accounts and prepared the following:

- GitHub Account
- Netlify Account
- Render Account
- React (Vite) frontend project
- Node/Express backend project
- A fully operational local build

Project Architecture Layout

```
my-project/
├── frontend/           # React/Vite App
│   ├── src/
│   ├── package.json
│   ├── vite.config.js
│   └── .env
└── backend/           # Node/Express API
    ├── routes/
    ├── models/
    ├── server.js
    ├── package.json
    └── .env
```

Deployment Guide Steps

Step 1 – Verify Everything Works Locally

Before initiating cloud deployment, make sure your application runs flawlessly on your local computer.

Start the backend:

```
npm run dev # or: npm start
```

Start the frontend:

```
npm run dev
```

Visit `http://localhost:5173` and verify:

-  Target data/weather endpoints load properly.
-  Database connects without authentication errors.
-  Network API requests return status code 200.
-  Browser developer tools console shows zero errors.

Golden Rule: Never attempt to deploy a broken application!

Step 2 – Push Your Project to GitHub

Commit and push your working codebase to your remote git repository. Both target hosting systems pull updates cleanly from here.

```
git add .  
git commit -m "Ready for deployment"  
git push origin main
```

Step 3 – Deploy the Backend to Render

1. Log in to your **Render Dashboard**.
2. Click **New +** → **Web Service**.
3. Connect your GitHub repository containing the backend code.
4. Configure the service using the specified parameters below:

Setting	Value
Name	<code>my-api</code> (or preferred choice)
Root Directory	<code>backend</code> (leave blank if repository root handles it)
Environment	<code>Node</code>
Build Command	<code>npm install</code>

Setting	Value
Start Command	<code>node server.js</code> (or <code>npm start</code>)

Click **Deploy Web Service**. After a few minutes, Render builds out a unique base address link:

```
https://my-api.onrender.com
```

Test It: Access your active link via browser. You should receive your standard live verification message (e.g., "Server Running!").

Step 4 & 5 – Handle Backend Environment Variables

Instead of exposing sensitive API connections directly inside codebases, values must be abstracted safely away.

Instead of hardcoding details like: `const apiKey = "123456";`

Leverage the environment setup: `const apiKey = process.env.API_KEY;`

Add Settings inside Render: Navigate through: *Render Dashboard* → *Your Service* → *Environment* → *Add Environment Variable*.

Key	Value
<code>MONGO_URI</code>	<code>mongodb+srv://...</code>
<code>WEATHER_API_KEY</code>	<code>abcdef...</code>
<code>JWT_SECRET</code>	<code>xyz123...</code>

Crucial Security Notice: Do NOT commit or upload your local `.env` file. Render automatically feeds these custom parameter pairs dynamically into your running production instance.

Step 6 – Deploy the Frontend to Netlify

1. Log into your **Netlify Dashboard**.
2. Click **Add New Site** → **Import from Git**.
3. Select your synced GitHub codebase repository.
4. Apply the mandatory build values for standard Vite structures:

Setting	Value
Build Command	<code>npm run build</code>
Publish Directory	<code>dist</code>

Click **Deploy Site**. Netlify outputs a secure preview URL like:

```
https://amazing-weather.netlify.app
```

Step 7 & 8 – Update Frontend Variables

Locally, your React system communicated with localhost: `VITE_SERVER_URL=http://localhost:3001`

In production, update this endpoint value to match your newly minted live Render address:

```
VITE_SERVER_URL=https://my-api.onrender.com
```

Add to Netlify Dashboard: Navigate to: *Site Settings* → *Environment Variables* → *Add Variable*.

Variable Key	Value
<code>VITE_SERVER_URL</code>	<code>https://my-api.onrender.com</code>

Note: Trigger a fresh redeploy on Netlify once updated so the engine embeds the updated target path variables.

Step 9 & 10 – Update Fetch Requests & Enable CORS

Avoid hardcoding server URLs in fetch parameters. Instead, leverage standard dynamic template literals:

```
// Dynamic, multi-environment routing request
fetch(`${import.meta.env.VITE_SERVER_URL}/weather`);
```

Enable Cross-Origin Resource Sharing (CORS)

Browsers block data delivery across contrasting domains unless specifically declared safe. In your backend directory, execute:

```
npm install cors
```

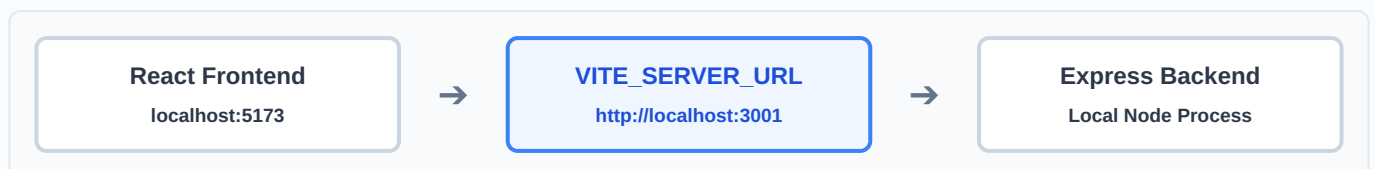
Then configure your `server.js` to grant access to the deployed Netlify site:

```
const cors = require("cors");

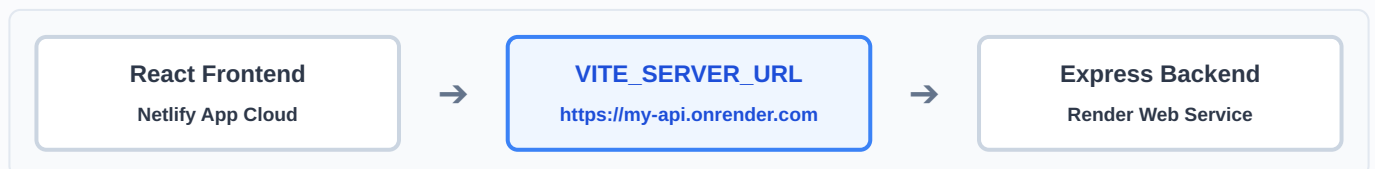
// Lock down authorization to your deployed frontend domain
app.use(cors({
  origin: "https://weather-app.netlify.app"
}));
```

Visualizing Architecture Changes

Local Development Cycle



Production Deployment Lifecycle



Common Troubleshooting Pitfalls

Mistake / Error	Actionable Solution
Using localhost post-deployment	Swap out static strings with <code>import.meta.env.VITE_SERVER_URL</code> values.
Missing Render key configurations	Ensure all key/value pairs from your local <code>.env</code> match inside the Render configuration tab.
CORS Errors blocking layout	Check that the middleware function is properly instantiated and targets your complete Netlify url layout.

Final Deployment Checklist

- Backend runs locally without failures.
- Frontend functions locally on port 5173.
- All changes committed and pushed to GitHub.
- Backend successfully configured on Render.
- Render Environment variables mirrored.
- Frontend successfully published via Netlify.
- Netlify context contains `VITE_SERVER_URL`.
- CORS framework configured to permit communication.